CVE-Guided Typestate Analysis for Recurring Vulnerability Detection

RUTVIJ WAGH, Indian Institute of Technology Guwahati, India
UDAYRAJ CHOUDHARY, Indian Institute of Technology Guwahati, India

A public vulnerability fix teaches a lifecycle rule: an object must not be released without resetting a caller-visible reference to it, or without detaching it from a chain it still belongs to. Existing general-purpose static analyzers do not check a specific project's own such conventions; they check their own stock rule sets instead. CVE-2022-4450 is a real case: `PEM_read_bio_ex` in OpenSSL frees two output buffers on one failure path without clearing the caller-visible pointers to them, letting a caller double-free. We present Tscк, which reads such a rule from a data file and checks it against a `tree-sitter` control-flow graph. On OpenSSL, Tscк flags `PEM_read_bio_ex`'s two outputs before the fix and clears both after it. It also reproduces an unrelated non-CVE fix the rule was never shown, and flags one site a later OpenSSL commit went on to fix independently. On the current 1,155-file `crypto/ssl/providers` tree it flags 10 sites: manual triage finds 2 potential recurrences whose failure branch is not reachable in normal builds, 8 harmless deviations, and 0 confirmed new vulnerabilities. None of three general-purpose open-source static analyzers we ran, with their stock configuration, distinguishes any of three historical CVEs from their own fix; a project-specific rule, not a sharper general one, closed that gap. A second, differently-shaped rule for the detach-before-release BIO-chain invariant behind CVE-2023-0215 reuses the same parser and control-flow graph and also distinguishes its own motivating fix. Its whole-tree run, though, flags 11 sites that are all false positives, because the rule has no notion of which released object is externally owned.

CCS Concepts: • **Software and its engineering** → **Software organization and properties**; • **Security and privacy** → *Software and application security*.

Additional Key Words and Phrases: static analysis, typestate analysis, control-flow graph, recurring vulnerabilities, OpenSSL, memory safety, output parameters, tree-sitter

1 Introduction

Suppose you have just read a public vulnerability fix for a C library you depend on: a one-line patch that frees a buffer and then clears the pointer to it, because the code path that frees it can be reached again before the function returns. You trust the fix. The natural next question is whether the same mistake, the same missing reset after the same kind of release, sits somewhere else in the same codebase, in a function the patch never touched.

Static analyzers already look for memory-safety bugs in C, and some are free and widely used: Clang Static Analyzer, Cppcheck, and Coccinelle semantic patches, among others. These tools ship with general rule sets built from common bug patterns across many projects. They stop short of one thing: a rule derived from a single project's own patch history, for the exact ownership or reset convention that patch teaches, checked everywhere that convention should hold in that project's current source.

We focus on a second problem that starts where general-purpose rule sets stop. Given one historical fix, can a rule extracted from it, with no function names baked in, be checked against a project's whole source tree? We ask three things of such a rule: (i) flag the function the fix came from before the fix, and clear it after; (ii) do the same for an unrelated fix nobody built the rule from; and (iii) say, for every site it flags on the project's current source, whether that site is conforming, a real violation, or explainable, with a witness a human can check by hand.

CVE-2022-4450 [National Vulnerability Database 2023a] is a real case. `PEM_read_bio_ex` in OpenSSL's `crypto/pem/pem_lib.c` frees two output buffers on one failure path (Figure 1) without

Authors' Contact Information: Rutvij Wagh, Indian Institute of Technology Guwahati, Guwahati, India; Udayraj Choudhary, Indian Institute of Technology Guwahati, Guwahati, India.

```
headerlen = BIO_get_mem_data(headerB, NULL);
*header = pem_malloc(headerlen + 1, flags);
*data = pem_malloc(len, flags);
if (*header == NULL || *data == NULL)
    goto out_free;
...
out_free: // *header, *data freed, never reset
    pem_free(*header, flags, 0);
    pem_free(*data, flags, 0);
end:
    ...
    return ret; // caller sees stale *header, *data
```

Fig. 1. `PEM_read_bio_ex` in OpenSSL `8e257b86e5` (`crypto/pem/pem_lib.c`, trimmed). The `out_free` path frees `*header` and `*data` but never resets either pointer before `return`. The fix, commit `63bcf189be`, adds `*header = NULL;` and `*data = NULL;` to this path.

clearing the caller-visible pointers to them; a caller that frees these outputs again after failure can double-free. The fix, commit `63bcf189be`, adds exactly two assignments.

Closest to our rule-as-data approach is VulPA [Cao et al. 2025a], which also derives a checkable rule from a known bug, but compiles it to an automaton checked by an interprocedural analysis with aliasing, on Java. Ours is intraprocedural, with no alias analysis (Section 8).

We present Tsck, a checker that reads a lifecycle rule from a data file and applies it to real C source through a parser and a per-function control-flow graph built with `tree-sitter`, searches for a path from each release of an output parameter to function exit that never resets it, and classifies each site with a witness path (Section 4). This paper implements two such rules against the same parser and CFG builder: Protocol B, for the output-pointer reset convention behind CVE-2022-4450, and Protocol A, for the detach-before-release BIO-chain convention behind CVE-2023-0215 [National Vulnerability Database 2023b] (**RQ5**, Section 5.8).

On OpenSSL, Tsck flags `PEM_read_bio_ex`'s two output parameters before the fix and clears both after it. It also reproduces an unrelated non-CVE fix the rule was never shown, and flags one site a later OpenSSL commit went on to fix independently (Section 5). On the current source tree it flags 10 sites. Manual triage finds 2 potential recurrences whose failure branch is not reachable in normal builds, 8 harmless deviations, and 0 confirmed new vulnerabilities. The lesson we did not expect going in: none of three general-purpose open-source static analyzers, run with their stock configuration, distinguishes any of the three CVEs we checked from its own fix (Section 5.5). A project-specific rule, not a sharper general one, is what closed the gap here. Protocol A also distinguishes its own motivating fix, but its whole-tree run finds 0 confirmed recurrences out of 11 flagged sites, all false positives from the rule's missing notion of object ownership: the rule-as-data architecture generalizes across invariant shapes even where a given rule's detection does not generalize across a whole codebase (Section 5.8).

Contributions. (1) A rule-as-data lifecycle invariant for output-pointer reset after release, independent of any function name (Section 3); (2) Tsck, which checks it on a real `tree-sitter` control-flow graph (Section 4); (3) an evaluation on six pinned OpenSSL snapshots and a four-tool comparison on three historical CVEs (Section 5); (4) a second, differently-shaped rule, Protocol A, showing where single-object generalization breaks down (Section 5.8); and (5) the released tool and data (Section 9).

2 Background

Output parameter. A C function can return a value through a parameter of pointer-to-pointer type, written `T **P`. The caller passes the address of its own pointer variable; the function writes a value into `*P` for the caller to read after the call returns. We write `*P` for the object the caller's pointer refers to, not for `P` itself.

Release and reset. A *release* of `*P` is a call that frees the object `*P` points to, for example `OPENSSL_free(*P)`. After a release, `*P` still holds the freed address unless something changes it. A *reset* of `*P` is an assignment that gives it a new value, most often `*P = NULL`. A dangling `*P` is dangerous when the caller can free or read it again (double free or use-after-free).

Typestate. Typestate analysis tracks not just an object's type but which operations are currently valid on it, as a small state machine, following the original formulation of typestate by Strom and Yemini [Strom and Yemini 1986]. For the invariant in this paper, `*P` moves through three states: *Valid* before any release, *Released* after a release and before a reset, and back to *Valid* (we call it *Reset*, to keep the release that caused it visible in our output) once a reset occurs. A *violation* is a program path on which `*P` can reach function exit while still in the *Released* state. CVE-2022-4450 [National Vulnerability Database 2023a] is exactly this.

Release site. We call the specific call expression that performs a release, together with its source location and the parameter `P` it releases, a *release site*. Section 5 counts release sites, not functions.

Rule as data. Existing patch-matching checkers, including the Part 1 baseline in this project (`typestate_check.py`), encode a rule as a sequence of string or syntax checks written for one named function. We instead write the rule once as data (Section 4.1), independent of any function name, and apply it to every function in a source tree that has a parameter of the tracked shape.

3 Problem Statement

We now state the problem precisely. Let f be a C function with a parameter P of pointer-to-pointer type or deeper. Let r be a *release site* inside f: a call that frees the object $*P$ points to (Section 2). A *reset* of $*P$ is an assignment to $*P$, or, when the rule allows it, passing P itself to a call that may reassign it. Let paths(r) be the set of control-flow paths from r to the exit of f, built from f's real parse tree (Section 4.2).

Goal. r is *conforming* if every path in paths(r) passes a reset of $*P$ before reaching exit. r is a *violation* if some path in paths(r) reaches exit without passing a reset. Given a rule (Section 4.1) and a C source tree, our goal is to compute this conforming/violation label, with a witness path, for every release site whose parameter matches the rule.

Why not grep for the release/reset pair? A regular expression, like the Part 1 baseline `typestate_check.py`, can check that a function's text contains both a release and a reset. It cannot tell whether the reset lies on *every* path from the release to exit or on only one branch, nor distinguish a reset in dead code; that needs a parse tree and a control-flow graph.

4 Approach

Figure 2 shows the steps of Tsck. The main idea is to read a lifecycle rule from a data file, rather than hard-coding it. Tsck checks that rule against a per-function control-flow graph built from the real parse tree of each C source file. We describe Protocol B here; Protocol A reuses the same parser and CFG (Section 5.8).

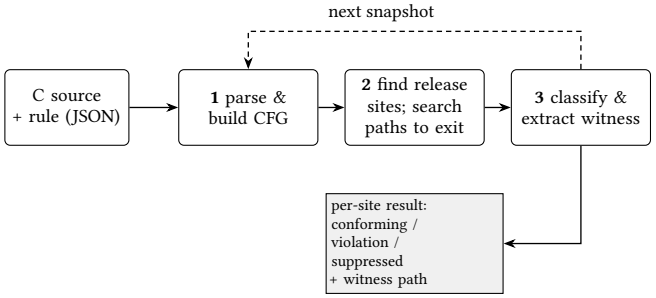


Fig. 2. Tsck takes a C source tree and a rule stored as data (Section 4.1). It parses every file once, finds every site that releases a tracked pointer, and searches forward in the control-flow graph for a path to function exit that never resets it. Each flagged site keeps a witness path. The outer loop (dashed) reruns this per pinned OpenSSL snapshot; nothing loops within one file's analysis.

4.1 The rule, as data

The rule for Protocol B is a JSON file, not a code path: it names no OpenSSL function. It tracks parameters of pointer-to-pointer type; a *release* is any call whose name matches the regular expression `(?i)free` and whose argument is `*P`; a *reset* is an assignment to `*P`, or, by a configurable flag, passing `P` itself to a later call. The invariant is: if a function releases `*P`, every path from that release to function exit must reach a reset first. We do not claim the idea of an output-parameter reset invariant as new; what we built is a parser and path-search engine that checks it against real source, rather than against one patch's text.

4.2 Step 1: Parsing and the control-flow graph

Tsck parses each C file with `tree-sitter` and `tree-sitter-c`. We do not claim parsing itself as new; everything from here on is. For each function, we build a control-flow graph whose nodes are simple statements, branch and loop conditions, labels, and join points; `goto` targets, including error labels such as `err:`, become real edges, so an error path that jumps past a reset is visible to the path search in Step 2. Three approximations keep the graph tractable and are stated as threats to validity (Section 7): a compound condition joined by `&&` or `||` is one node, without separate short-circuit edges; a `#if`/`#ifdef` block is treated as a branch, keeping both variants rather than resolving the macro; and macro bodies are not expanded, so a macro call is treated as an ordinary call expression.

4.3 Step 2: Release sites and the path search

For every parameter `P` of pointer-to-pointer type or deeper, Tsck scans each control-flow node for calls matching the release pattern and for assignments matching the reset pattern. From every release site, it searches the control-flow graph forward, breadth-first, for a path to function exit. The search stops following a branch once it crosses a reset, and the release's own dereference of `*P` licenses one pruning rule: because the release itself has already dereferenced `*P`, `P` is non-null on every path after it, so a later branch that is live only when `P == NULL` is dead code and is not explored. If the search reaches function exit without crossing a reset, the site is a candidate violation and the path taken becomes its witness.

Two refinements, both chosen after reading real output, narrow the candidates: *deref-after-release suppression* drops a site where `*P` is read again before any reset (a partial release and reuse, not a dangling pointer), and the `NULL`-guard pruning above was added after one infeasible-path false positive. Both are unit-tested and listed in each result.

4.4 Step 3: Classification and witness extraction

Each release site is labeled *conforming* if every path from it to function exit passes a reset; *violation* if some path reaches exit without one; or *suppressed* if the path rule would flag it but one of the two refinements in Step 2 applies. Suppressed sites are reported with their reason, never dropped. For a violation, Tsck records the witness path's source lines so a reader can check it without re-running the tool. `compare.py` diffs two runs of the same files at different revisions and reports exactly which sites a given commit removed or added. Section 5 uses this to measure whether a real fix changes only the sites its own diff touches.

5 Evaluation

We ask five questions.

- **RQ1** Does Tsck (the invariant-based Protocol B engine) detect the motivating CVE on real OpenSSL source, distinguish the fix from the bug, and generalize to a fix it was not derived from?
- **RQ2** Do general-purpose open-source static-analysis tools already catch the bugs a source-pattern baseline catches, on the same three CVEs?
- **RQ3** Does a rule written from one CVE flag a site that OpenSSL fixed only later, without the rule being changed after the fact?
- **RQ4** What does Tsck cost in time?
- **RQ5** Does Tsck's rule-as-data design extend to a second, differently-shaped invariant (Protocol A, CVE-2023-0215), and does that second rule generalize the same way Protocol B's did?

5.1 Subjects

We select three historical OpenSSL CVEs and one non-CVE fix, each a pinned vulnerable/fixed commit pair with a publicly documented root cause (so the label is not our judgment), source at both revisions in the same file, and at least one case not used to derive the rule (**RQ3**). CVE-2022-4450 (`PEM_read_bio_ex`, fix `63bcf189be`) is the source of Protocol B. CVE-2023-0215 (`BIO_new_NDEF`, fix `8818064ce3`) is the second baseline CVE (**RQ2**) and the source of Protocol A (**RQ5**). CVE-2024-4741 (`SSL_free_buffers`, fix `b3f0eb0a29`) is in the baseline comparison only: it needs a state guard neither rule can express, and we report that scope rather than drop the case. The `d2217c88df` KDF fix (not a CVE, a different author, a year before CVE-2022-4450) is the out-of-sample case.

5.2 Baselines

The *Part 1 pattern baseline* is `typestate_check.py`, the project's own string/regex patch matcher, included as the simplest thing that could work. *Clang Static Analyzer 18* ran via `scan-build-18` on each revision's affected translation unit with its generated build flags. *Coccinelle 1.1.1* ran five pre-existing semantic patches (not written for these CVEs) for release/use, repeated release, release-after-transfer, and allocate-then-return. *Cppcheck 2.13.0* ran with `–enable=warning,style,performance, portability –inconclusive` (C11/unix64). None was tuned toward these CVEs. Because Protocol B has no rule for two of the three CVEs, this compares general tools against the baseline, not against Tsck. For each tool and revision we manually read whether the output distinguishes vulnerable from fixed.

5.3 Setup

Tsck (Protocol B) ran on Python 3.13, `tree-sitter` 0.26.0, `tree-sitter-c` 0.24.2 (Linux, WSL2). The **RQ2** baselines ran on a separate machine (Python 3.12.3, Clang 18.1.3, Coccinelle 1.1.1 on OCaml 4.14.1, Cppcheck 2.13.0); executable hashes and frozen rule files are recorded with the data,

Table 1. Tscк (Protocol B) on six pinned OpenSSL snapshots (`crypto/`, `ssl/`, `providers/`). A *release site* is a call that frees `*P` for a pointer-to-pointer parameter `P`. The two fixes each remove exactly their own violations and add none.

Snapshot	C files	Conforming	Violation	Suppressed
CVE-2022-4450 pre-fix (`8e257b86e5`)	1001	99	7	2
CVE-2022-4450 fix (`63bcf189be`)	1001	101	5	2
KDF fix parent (`29a27cb2c5`)	1005	93	12	2
KDF fix (`d2217c88df`, not a CVE)	1005	97	8	2
OpenSSL 3.0.0 (`89cd17a031`)	999	92	10	2
master (`949fe8b5f4`, 2026-10-30)	1155	132	10	2

Table 2. Detection on the vulnerable/fixed pair for each CVE: whether the tool's output distinguishes the two revisions for the CVE's own mechanism. ✓ = distinguishes vulnerable from fixed; ✗ = does not. The Part 1 pattern baseline is not Tscк; see Section 5.2.

Tool	2022 4450	2023 0215	2024 4741
Part 1 pattern baseline	✓	✓	✗[a]
Clang Static Analyzer 18	✗	✗	✗
Coccinelle 1.1.1	✗	✗	✗
Cppcheck 2.13.0	✗	✗	✗

and no measurement mixes the two environments. For **RQ4**, a second author reran `run_scan.sh` on a third machine (Python 3.11.6, same `tree-sitter` versions) under `/usr/bin/time -v`. Protocol A's runs (**RQ5**) were reproduced on a fourth environment (`tree-sitter` 0.26.1) with identical counts.

5.4 RQ1: Does Tscк detect and generalize the motivating CVE?

Table 1 shows Tscк on the motivating function and on the whole tree. At the vulnerable revision, Tscк flags `PEM_read_bio_ex`'s `*header` and `*data` as violations (witness lines 991–998); at the fix, both sites are conforming. The same fix removes exactly these two violations from the whole-tree count (99 → 101 conforming, 7 → 5 violation) and adds none.

Observation 1. Tscк is not limited to the function it was derived from: the unrelated, non-CVE `d2217c88df` KDF fix removes exactly the four violations its own commit changes (`pbkdf1`, `pbkdf2`, `pkcs12kdf`, `scrypt`, each `*buffer`) and adds none, with the rule unchanged from the CVE-2022-4450 derivation.

Observation 2. On current master, Tscк flags 10 sites; manual triage against each site's callers found 2 potential recurrences whose failure branch is not reachable in normal builds, 8 convention deviations with no caller impact, and 0 confirmed new vulnerabilities. (Two of the eight were initially labelled "unclear" pending a caller read; Section 7 reports that re-check.) We report this plainly: the contribution here is a bounded rule that reproduces two real fixes without retuning, not a new bug.

5.5 RQ2: Do general-purpose tools already catch these bugs?

The columns correspond to CVE-2022-4450 [National Vulnerability Database 2023a], CVE-2023-0215 [National Vulnerability Database 2023b], and CVE-2024-4741 [National Vulnerability Database 2024], respectively. [a]On the corrected CVE-2024-4741 pair (`ec559c2a6c/b3f0eb0a29`) the Part 1 baseline alerts on both sides; it is quiet only on the later hardening commit `2d05959073`, so it matches that pattern, not the advisory's state-guard fix. Table 2 compares four tools on six pinned revisions (three CVEs, vulnerable and fixed side). A Clang positive control, run on a synthetic use-after-free, produced the expected diagnostic with the same configuration, confirming the analyzer was active for the clean runs.

Observation 3. None of three general-purpose open-source tools, run with their stock rules and no CVE-specific tuning, distinguishes the vulnerable revision from the fixed revision for any of the three mechanisms. Coccinelle's generic release-event rule fires near `BIO_new_NDEF` in both revisions, but does not inspect `BIO_pop` or the linked-BIO state, so it does not change between versions; Cppcheck's output on the three files is unrelated style diagnostics, identical in count before and after each fix; Clang emits no diagnostic in any of the six runs.

This does not show Tscк beats these tools: Protocol B (Tscк itself) does not implement a rule for two of these three mechanisms (Section 5.1). It shows that a source check specific to one ownership or reset convention is needed at all, because the general-purpose tools' stock configuration does not substitute for it.

5.6 RQ3: Does the rule generalize to an unseen later fix?

The tree at `d2217c88df` still flags `pvk_set_membuf` in `pvkkdf.c`. OpenSSL fixed it one day later, in `a632bfaa4e`, a direct child of `d2217c88df`, with the same `*buffer = NULL` change as the other four KDF sites.

Observation 4. Tscк, run unchanged from its CVE-2022-4450 derivation, flags a site that a different, later commit by OpenSSL went on to fix. This is one site, not a rate, and we report it as a single encouraging case, not as a time-travel precision or recall figure: a systematic time-travel evaluation over more sites is future work.

5.7 RQ4: What does Tscк cost in time?

Observation 5. The independent rerun (Section 5.3) reproduced every count in Table 1 exactly. Across five full-tree scans of 999–1,155 files plus two single-function checks, CPU time was under 48 seconds (27.17 s user, 20.34 s system) of 5 min 31 s wall-clock; the rest is one-time network latency from a partial clone of OpenSSL. The CPU figure still includes `git` sparse-checkout cost, which we did not instrument separately.

5.8 RQ5: Does a second, differently-shaped rule generalize the same way?

Protocol A checks a different invariant shape than Protocol B: establish-then-required-detach-before-release on a local variable rather than release-then-required-reset on a parameter, with a syntactic alias rule for the common `x = BIO_push(v, ...)` idiom (Section 5.1). It reuses Protocol B's `tree-sitter` parser and CFG; only the event rule and path search are new. We ran it two ways, both independently reproduced on a separate machine (Section 5.3).

Observation 6. On `BIO_new_NDEF`, Protocol A flags `asn_bio` as a violation before the CVE-2023-0215 fix and as conforming after it, the same pre/post-fix distinction Protocol B makes for CVE-2022-4450. The rule-as-data design extends to a second invariant shape without changing the parser or the CFG builder.

On the same master snapshot, Protocol A flags 11 sites; we checked every one against the pinned source (`results/protocol_a_triage.md`).

Observation 7. 0 of the 11 whole-tree violations are confirmed or plausible recurrences. One is a tool limitation: a real `BIO_pop` is guarded by a boolean flag the path search cannot correlate with the branch that set it. The other 10 release a chain built entirely from freshly-allocated local objects (two behind a push whose failure branch is unreachable once its first argument is proven non-`NULL`). None splices in an externally-owned BIO – the shape that makes CVE-2023-0215 dangerous – and the rule has no notion of ownership to tell them apart.

This confirms the gap named in Section 8: without ownership or escape tracking, a single-object rule cannot generalize a BIO-chain invariant across a codebase, even though the same architecture generalized a release/reset invariant in **RQ1**.

6 Discussion

Did Tscк find a new bug? No. The two potential-recurrence sites on master (`X509_PUBKEY_set`, `ossl_ml_kem_i2d_pubkey`; Observation 2) share one trigger: the tracked object's state must change between two reads that a single straight-line call treats as equal, which only an unsynchronized second thread racing the same object can cause. Re-checking against the pinned source (`results/reachability_rq1_candidates.md`) also corrected our original note, which blamed a lock/atomic-op failure in `EVP_PKEY_up_ref`; every platform variant of `CRYPTO_UP_REF` here returns `true`, so that path is dead code. No in-tree caller creates the precondition, and we built no multithreaded fault-injection test, so both remain candidates, not findings.

Is this just matching one patch? The rule names no OpenSSL function, yet detects the unrelated `d2217c88df` fix unchanged (Observation 1) and a sibling site fixed later (Observation 4) – two data points, not a rate. Independently of the tool, a commit-message search of OpenSSL's history (`results/historical_recurrence_search.md`) found ten further, independently-authored fixes of this shape across nine files (2023 to six weeks before our snapshot), mostly struct-field and local-variable resets that neither engine tracks. This corroborates the convention's generality without being a tool finding.

Why a custom engine? Table 2 is the attempt to use existing tools: with stock configurations none separates any of the three CVEs from its fix (Observation 3), because their rule sets do not encode this project's conventions.

False positives and scope. Across all six snapshots, 1 of 18 flagged Protocol B sites is a confirmed false positive (`load_common`, which releases only a structural reference); two more are correctly suppressed. For CVE-2023-0215, the Table 2 distinction comes from the Part 1 baseline, not Tscк; for CVE-2024-4741 no tool, including the baseline, distinguishes the corrected pair. Protocol A covers CVE-2023-0215's own pair but its whole-tree run is 0 for 11 (Section 5.8); CVE-2024-4741 needs a state-guard rule neither shape expresses. Finally, the 91.7% master conformance rate is not a precision number: it holds only under the name-based release heuristic, and sites that heuristic cannot see are absent from both numerator and denominator.

7 Threats to Validity

Internal. Tuning on seen data: the deref-after-release suppression and `NULL`-guard pruning (Section 4.3) were added after reading OpenSSL output, so the master numbers in Table 1 are not a blind test of them; the rule itself (Section 4.1) was not changed after the KDF result. *Name-based release detection:* `(?i)free` cannot see reference counts, which produced the `load_common` false positive. *Parser coverage:* about 39% of files have a `tree-sitter` parse error, almost always an unexpanded macro; only 7–8 release sites per snapshot sit in such functions and no reported violation is among them, but a site in an unparseable function is invisible to Tsck. *Our own triage:* the master labels (Observation 2) were assigned by the tool's builders. A generative AI assistant not involved in building the rule re-derived the reachability of the 2 potential-recurrence sites (correcting one error in our note) and re-checked the 2 formerly "unclear" sites, which reclassify as convention deviation once their one in-tree caller, `apps/cmp.c`, is read. An independent human second labeller remains future work.

External. Protocol B's generalization rests on two data points, not a detection rate; Protocol A's whole-tree run is 0 for 11 (Observation 7), a real negative result showing that a single-object detach rule without ownership tracking does not generalize the way Protocol B did. The six snapshots share most long-lived functions, so the effective sample is closer to 20 distinct release sites than to 20×6. All subjects are OpenSSL, and Tsck tracks only pointer-to-pointer parameters, not struct-member out-parameters such as `ctx->buf`. More historical pairs, a second project, and an ownership/escape analysis are future work.

Construct. A violation is a source-pattern property, not a proven vulnerability: both potential-recurrence sites need a data race no in-tree caller creates (Section 6). A ✗ in Table 2 means no diagnostic for that translation unit and configuration (Clang analyzed only the affected unit), not absence of a bug. And RQ2's baseline is the Part 1 pattern matcher, not Protocol B; we do not credit Tsck with that result.

8 Related Work

Rule-guided recurring vulnerability detection. Closest to us is the work of Cao et al. [Cao et al. 2025a], VulPA, which also derives a checkable rule from a known bug and searches a codebase for other violations. Their rule language, VPDL, compiles to an automaton and can bind several correlated variables at once, for example checking that the same object reached both a permission check and a sensitive call; an IFDS-based interprocedural analysis then walks the whole program. VulPA targets Java and reports 90 confirmed new findings across 26 applications and 34 source CVEs. Our rules are intraprocedural, with no alias analysis, and evaluated on far smaller scale. Protocol A (Section 5.8) is direct evidence of this gap: its whole-tree run is 0 for 11 precisely because it lacks the multi-object, ownership-aware correlation VulPA has.

Clone-based recurring vulnerability detection. Cao et al. [Cao et al. 2025b] study how well existing recurring-vulnerability detectors (VUDDY, MVP, MOVERY, V1scan, FIRE) find code *reused* from a known-vulnerable function. They show these detectors do much worse on syntactically or semantically modified clones than on exact or renamed ones. Their notion of *recurring* is a vulnerable function's code reappearing, by reuse, elsewhere; ours is a different function independently repeating the same mistake, with no clone relationship (e.g., the four KDF functions in Observation 1). Tan et al. [Tan et al. 2025] address a failure mode specific to clone-based detectors: code that is similar to a vulnerable function but has itself already been patched, and which clone-based tools then misflag. Tsck checks control flow, not similarity, so it avoids this false-positive source (though not those in Section 7).

Locating when a vulnerability was introduced. Guo and He [Guo and He 2026] identify the commit that first introduced a known vulnerability, by extracting the vulnerable/patched diff's statement-level patching pattern and matching it backward through commit history. That asks which commit introduced a known bug, not which other functions share its pattern now; we take each fix commit as given.

Patch porting. Wu et al. [Wu et al. 2025] port an existing, known patch to other versions or branches of the same project, combining semantic and syntactic matching with an LLM. This assumes the target location is known; we find locations not yet known to need the fix, so the two could be complementary (not evaluated).

9 Conclusion

A vulnerability fix is evidence of a convention a project expects its own code to follow, not just a patch to one function. CVE-2022-4450 shows this concretely: `PEM_read_bio_ex` frees two output buffers on a failure path and leaves the caller-visible pointers dangling, and the same release-without-reset shape recurs, unrelated to that fix, in four KDF functions a year earlier. General-purpose static analyzers, run with their stock rule sets, catch none of the three historical CVEs we checked.

We presented Tsck, which checks a lifecycle rule stored as data against a real `tree-sitter` control-flow graph. Protocol B separates CVE-2022-4450 from its fix, reproduces the unrelated KDF fix, and flags a sibling site OpenSSL later fixed. On OpenSSL's current 1,155-file tree it flags 10 sites, of which manual triage confirms 0 new vulnerabilities, 2 potential recurrences needing reachability review, and 8 harmless deviations. Protocol A, for CVE-2023-0215's BIO-chain invariant, reuses the same parser and CFG and separates its own fix, but its whole-tree run is 0 for 11, for want of ownership tracking: the architecture generalizes across invariant shapes even where one rule's detection does not. We make no claim beyond this: two rules, one project, with the threats in Section 7. An ownership/escape analysis for Protocol A, and a broader historical evaluation, are future work.

Data Availability

This is a named submission (confirmed with the course instructor, 2026-11-13), so the replication package below is the real, public repository, not an anonymized mirror: https://github.com/PASS-IITG/cs5187-term-project-dauntlesscrux.

It contains Tsck's source (`src/tsck/`) and 19 engine unit tests; both rules as data (`specs/outparam_reset.json`, `specs/bio_chain_pop.json`) with Protocol A's checker and regression tests; the pinned OpenSSL SHAs and scan scripts (`run_scan.sh`, `run_protocol_a.sh`); and, under `src/tsck/results/`, the unedited logs, JSON results, tool versions, timing rerun, manual triage for both protocols, the reachability and caller re-checks, the commit-history search, and the tool-comparison evidence behind Table 2. `make check` reruns the offline checks; `make scan` reruns every OpenSSL scan from a clean checkout.

Author Contributions

Rutvij Wagh (230101113), Protocol B lead: root-cause analysis of CVE-2022-4450; formulation of the output-pointer reset invariant; development of the Protocol B rule specification and checker (`outparam.py`); validation against the vulnerable and fixed OpenSSL revisions; evaluation of the unrelated KDF fixes and the Protocol B results; analysis of flagged sites and their false positives.
Udayraj Choudhary (230101107), Protocol A lead: investigation of CVE-2023-0215 and the BIO-chain use-after-free; AddressSanitizer reproduction and vulnerable/fixed revision analysis; formulation of the BIO push/pop invariant; oversight and validation of the Protocol A specification,

checker (`protocol_a.py`), and regression tests; investigation of the whole-tree results and its 11 false positives. **Joint:** selection of the project topic and OpenSSL as the research subject; design of the shared tree-sitter parsing and CFG architecture; selection of research questions and evaluation methodology; integration of both typestate protocols; comparison with general-purpose static-analysis tools; literature review and interpretation of results; paper preparation, proofreading, debugging, and final validation; preparation of the final presentation and demonstration. Both authors reviewed and take responsibility for the full paper and replication package, including the sections produced with generative-AI assistance disclosed below.

Use of Generative AI

For Protocol B, the authors designed the rule and produced all OpenSSL builds and measurements; a generative AI assistant (Claude) helped draft prose from those results. For Protocol A, the assistant helped formulate and implement the checker from the authors' CVE-2023-0215 notes, ran it on an independently cloned OpenSSL tree, and produced the initial triage of its 11 whole-tree findings; the authors reproduced this run on a separate machine (Section 5.3). The assistant also wrote the analyses in `results/reachability_rq1_candidates.md`, `results/unclear_sites_recheck.md`, and `results/historical_recurrence_search.md`. In the final revision, ChatGPT (GPT-6) verified bibliographic metadata and ACM CCS identifiers and edited LaTeX/BibTeX files, and Claude condensed the text to fit the page limit without changing any result. No AI tool altered recorded experimental outputs. All source excerpts, logs, and reasoning are in the replication package, and the authors take responsibility for every claim.

References

Liqing Cao, Haofeng Li, Chenghang Shi, Jie Lu, Haining Meng, Lian Li, and Jingling Xue. 2025a. VulPA: Detecting Semantically Recurring Vulnerabilities with Multi-object Typestate Analysis. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2430–2453. Accessed: 2026-11-13. doi:10.1145/3729378

Yiheng Cao, Susheng Wu, Ruisi Wang, Bihuan Chen, Yiheng Huang, Chenhao Lu, Zhuotong Zhou, and Xin Peng. 2025b. Recurring Vulnerability Detection: How Far Are We? *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 573–595. Accessed: 2026-11-13. doi:10.1145/3728901

Qixuan Guo and Yongzhong He. 2026. Accurate Identification of the Vulnerability-Introducing Commit Based on Differential Analysis of Patching Patterns. Proceedings of the Network and Distributed System Security (NDSS) Symposium 2026. Paper s140; 23–27 February 2026; accessed 2026-11-13. doi:10.14722/ndss.2026.230140

National Vulnerability Database. 2023a. CVE-2022-4450 Detail. NIST National Vulnerability Database. Accessed: 2026-11-13. https://nvd.nist.gov/vuln/detail/CVE-2022-4450

National Vulnerability Database. 2023b. CVE-2023-0215 Detail. NIST National Vulnerability Database. Accessed: 2026-11-13. https://nvd.nist.gov/vuln/detail/CVE-2023-0215

National Vulnerability Database. 2024. CVE-2024-4741 Detail. NIST National Vulnerability Database. Accessed: 2026-11-13. https://nvd.nist.gov/vuln/detail/CVE-2024-4741

Robert E. Strom and Shaula Yemini. 1986. Typestate: A Programming Language Concept for Enhancing Software Reliability. *IEEE Transactions on Software Engineering* 12, 1 (1986), 157–171. Accessed: 2026-11-13. doi:10.1109/TSE.1986.6312929

Zixuan Tan, Jiayuan Zhou, Xing Hu, Shengyi Pan, Kui Liu, and Xin Xia. 2025. Similar but Patched Code Considered Harmful: The Impact of Similar but Patched Code on Recurring Vulnerability Detection and How to Remove Them. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Ottawa, ON, Canada, 2445–2457. Accessed: 2026-11-13. doi:10.1109/ICSE55347.2025.00110

Susheng Wu, Ruisi Wang, Yiheng Cao, Bihuan Chen, Zhuotong Zhou, Yiheng Huang, Junpeng Zhao, and Xin Peng. 2025. Mystique: Automated Vulnerability Patch Porting with Semantic and Syntactic-Enhanced LLM. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 130–152. Accessed: 2026-11-13. doi:10.1145/3715718